EDUCORE

Backend Tech Stack Recommendation

School Management SaaS — Targeting 500+ Schools in Bangladesh

500+	1.5M+	Month-1	VPS+S3
TARGET SCHOOLS	MAX USERS	FEE SPIKE EVENT	HOSTING MODEL

BACKEND FRAMEWORK

NestJS — Recommended

TypeScript-first, modular, built-in DI, guards, interceptors. Shares TypeScript with your React frontend — one language across the full stack.

- ✓ Shares types with React frontend via shared DTOs
- ✓ Built-in module system matches your 20-module design
- ✓ Guards make multi-tenant RBAC clean and declarative
- ✓ Microservices-ready when you need to split later
- ✓ Best ecosystem for JWT, queues, WebSockets, cron jobs

Laravel — Avoid

Excellent for monoliths. But PHP = different language from your React frontend. Slower under heavy concurrent load vs Node. Not ideal for real-time features.

- ✗ PHP type system doesn't share with TypeScript frontend
- ✗ Weaker on concurrent real-time (WebSocket, SSE)

Golang — Overkill now

Fastest raw performance. But steep learning curve, no ORM as mature as TypeORM/Prisma, and premature for this stage. Revisit at 2000+ schools.

- ✗ Ecosystem less mature for rapid feature dev
- ✗ No shared types with React; harder to hire for

DATABASE

PostgreSQL — Recommended

Gold standard for relational SaaS. Row-level security, JSONB columns for flexible fields, excellent indexing, native UUID, and full ACID transactions for fee payments.

- ✓ ACID transactions — critical for fee collection & payroll
- ✓ Row-Level Security (RLS) for tenant isolation at DB level
- ✓ JSONB for flexible custom fields per school
- ✓ Handles 500+ schools on one instance easily

MongoDB — Wrong fit

No ACID transactions across collections. Fee payments, payroll, and ledgers need strict consistency. School data is deeply relational — students → classes → subjects → marks → fees.

- ✗ No multi-document transactions (until v4, still weak)
- ✗ Relational joins are expensive and awkward

MySQL — Acceptable

Works, but PostgreSQL is strictly better: native JSON, better constraints, superior indexing, window functions, and a cleaner ORM experience with Prisma/TypeORM.

- ✗ Inferior JSON support vs Postgres JSONB
- ✗ No Row-Level Security natively

THE CRITICAL QUESTION — MULTI-TENANT STRATEGY

Your plan (one VPS per school) is the wrong approach at 500+ schools. That is 500+ servers to manage, patch, monitor, and pay for — a DevOps nightmare. Here is the right model.

Shared DB + Schema Isolation

Use this

One PostgreSQL instance. Each school gets its own PostgreSQL schema (like a namespace). All schools share the same binary but their data is fully separated at the schema level.

- ✓ One server, 500+ schools — manageable DevOps
- ✓ Full data isolation — school A cannot see school B
- ✓ Easy per-school backup (dump one schema)
- ✓ Migrations run once, apply to all schemas
- ✓ Custom domain = just a CNAME, same server

One VPS Per School

Don't do this

500 schools = 500 servers. Security patches × 500. Database migrations × 500. Monitoring × 500. Costs spiral. One school's server goes down at midnight — you wake up.

- ✗ 500+ servers to maintain, patch, and monitor
- ✗ Feature deployment = 500 separate deploys
- ✗ Cost: 500 VPS × BDT ~800/mo = BDT 4 lakh/month
- ✗ When a school wants a custom domain, far harder

Row-Level Isolation (school_id)

Simpler but weaker

One schema, all schools in same tables, every row has school_id. Simpler to build, but a missed WHERE clause leaks data across schools. Not suitable for 500+ schools handling fee data.

- ✗ One missed WHERE leaks data between schools
- ✗ Per-school backup is complicated

HANDLING THE MONTH-1 FEE PAYMENT SPIKE

When 500 schools × 1500 students each hit "Pay fees" on the 1st of the month, that's 750,000 near-simultaneous requests. Raw synchronous DB writes will kill your server. Here is what you need.

BullMQ Job Queue (Redis-backed)

Payment requests don't hit PostgreSQL directly. They enter a BullMQ queue. Workers process them in controlled batches (e.g. 100 concurrent). The API returns "Payment received — processing" immediately. The student sees success in seconds. DB never gets slammed.

Redis for Session, Rate Limiting, and Cache

Store JWT sessions in Redis (not DB). Cache fee structures, class lists, and dashboard stats in Redis with TTL. Rate-limit bKash/Nagad webhook endpoints. Dramatically reduces DB reads during peak time.

PostgreSQL Connection Pooling (PgBouncer)

Postgres can't handle 10,000 simultaneous connections. PgBouncer sits between NestJS and Postgres and pools connections to ~200 max. Essential for any SaaS at this scale.

FILE STORAGE — YOUR QUESTION ABOUT DOCUMENTS

Cloudflare R2 (not AWS S3)

For Bangladesh, Cloudflare R2 is the right choice over AWS S3. R2 has zero egress fees (S3 charges per GB downloaded — adds up fast for report PDFs and documents). R2 is S3-compatible so your SDK code is identical. Store: student photos, admission documents, result PDFs, certificates, assignment uploads, teacher NID scans.

Storage Folder Structure Per School

Bucket → /school-slug/ → /students/ /documents/ /results/ /certificates/ /assignments/ — each school's files are namespaced. Signed URLs for access (files never public). Auto-expiring download links for report cards shared with parents.

SUBDOMAIN + CUSTOM DOMAIN ROUTING

Nginx on One VPS Handles All Routing

One Nginx reverse proxy reads the Host header. "school-a.educore.com.bd" → extracts slug "school-a" → passes to NestJS as X-Tenant header. NestJS middleware reads the header, sets the PostgreSQL schema to "school_a". Custom domains: school provides CNAME record pointing to your server IP. Nginx catches it, maps it to their slug. Zero new servers needed.

FINAL RECOMMENDED STACK

BACKEND	DATABASE	TENANCY	CACHE + QUEUE
NestJS + TypeScript	**PostgreSQL**	**Schema-per-school**	**Redis + BullMQ**
Modular, shared types with React, strong ecosystem	Schema-per-tenant isolation, ACID for payments	Full isolation, one server, easy backup	Session cache + payment queue for spike handling

DB POOL	FILE STORAGE	ORM	ROUTING
PgBouncer	**Cloudflare R2**	**Prisma**	**Nginx**
Handles 500+ school concurrent connections	S3-compatible, zero egress cost, BD-friendly	Type-safe, schema migrations, multi-schema support	Subdomain + custom domain via Host header

HOSTING

2–3 VPS Total

App server + DB server + Redis. Not per school.

PDF GEN

Puppeteer (queue)

Async job — never block API for report cards

PAYMENTS

bKash + Nagad API

Via queue workers, not synchronous API handlers

SMS

SSL Wireless BD

BD-specific, bulk SMS for attendance/fees